Recurrent Neural Networks

Chapter 15: **Processing Sequences Using RNNs and CNNs**
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

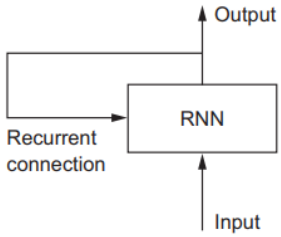
Recurrent Neural Networks (RNNs)

- RNNs are a family of neural networks specialized for processing sequential data.
- It processes sequences by iterating through the sequence elements and maintaining a state containing information relative to what it has seen so far; it maintains a memory state.
- In effect, an RNN is a type of neural network that has an internal loop; each training example / input is not processed in a single step; rather, the network internally loops over sequence elements.
- Hence at every time step the network has information not only from the current inputs but also from the history of the inputs.
- Mathematically,

$$h_t = af(x_t, h_{t-1})$$

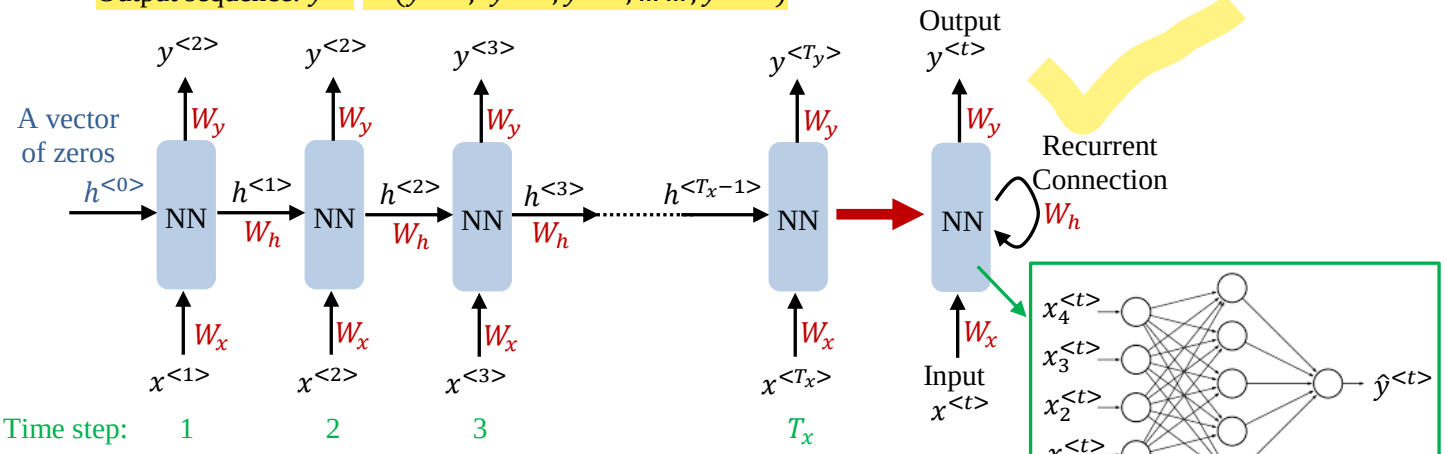
Where x_t is the current input, h_{t-1} is the previous memory, and h_t is the updated memory. af is the activation function.

- The state of the RNN is reset between processing two different, independent sequences (training examples).



Example: Consider reading a sentence from left to right. At each time step t, the network outputs a probability (or prediction) corresponding to each input word. Therefore, for each sentence, the number of inputs (words) is equal to the number of outputs.
For the j^{th} training example, let the input sentence length be T_{x_j} and output sequence length be T_{y_j}. To simplify notation, we omit the example index j. Then, $x^{<t>}$ denotes the word at time step t.
Hence for the j^{th} example with $T_x = T_y$: number of input = number of output

Input sequence: $x^{(j)} = (x^{<1>}, x^{<3>}, x^{<3>}, \dots\dots, x^{<T_x>})$
Output sequence: $y^{(j)} = (y^{<1>}, y^{<3>}, y^{<3>}, \dots\dots, y^{<T_y>})$



$$h^{<0>} = 0$$
$$h^{<1>} = af_h(h^{<0>}, x^{<1>}) = af(W_h h^{<0>} + W_x x^{<1>} + b_h)$$
$$y^{<1>} = af_y(h^{<1>}) = af(W_y h^{<1>} + b_y)$$

- In general, for time step t: $h^{<t>} = af(h^{<t-1>}, x^{<t>}) = af(W_h h^{<t-1>} + W_x x^{<t>} + b_h)$
- $h^{<t>}$ is the internal, memory state passed on to the next time stepg.
- First time step: First word $x^{<1>}$ is fed to the neural network (NN) to produce the output $\hat{y}^{<1>}$.
- Second time step: Second word is fed to the NN, along with the previous information in the form of the input $h^{<1>}$, and produces an output $\hat{y}^{<2>}$.
- The process continues until the last word in the sentence has been processed.
- $h^{<t>}$ is the state which is updated as the sequence is progressed.

- At each time step the memory state $h^{<t>}$ is a function with weights of external input $x^{<t>}$ as well as the previous memory state $h^{<t-1>}$, and $h^{<t>}$ is a recurrence relation given as:

$$h^{<t>} = af(h^{<t-1>}, x^{<t>})$$

- All the time steps share the same weight matrices, hence the process can be viewed as a single NN with a self loop.
- Some output $\hat{y}^{<t>}$ can also be produced using this cell state as required by a particular application.

Memory Cell

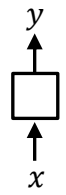
- Since the output of a recurrent neuron at time step t is a function of all the inputs from previous time steps, it can be viewed as a form of memory.
- A part of a neural network that preserves some state across time steps is called a memory cell (or simply a cell).
- In a basic RNN, the hidden state acts as this memory, but it is typically limited to capturing short-term dependencies due to issues like vanishing gradients.
- To model long-term dependencies, more advanced architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are used.

Types of RNNs Based on Input–Output Sequences

- The number of inputs and outputs per training example can vary depending on the application.
- Based on this, different RNN architectures are used.

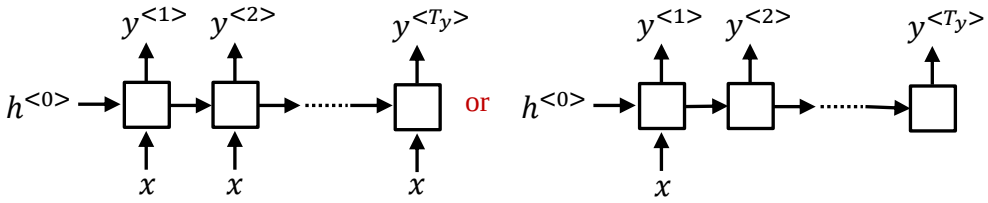
One-to-one (Not an RNN)

- A standard neural network with one input and one output
- Example: Image classification
  Input: Image
  Output: Class



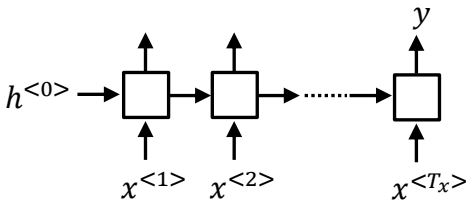
One-to-many

- Single input, sequence output
- Vector-to-sequence model
- Example: Image captioning
  Input: Image of a cat
  Output: 'Cat'



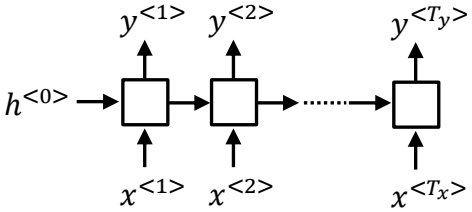
Many-to-one

- Sequence input, single output
- Sequence-to-vector model
- Example: Sentiment analysis
  Input: 'There is nothing to like in this book.'
  Output: 'Negative'



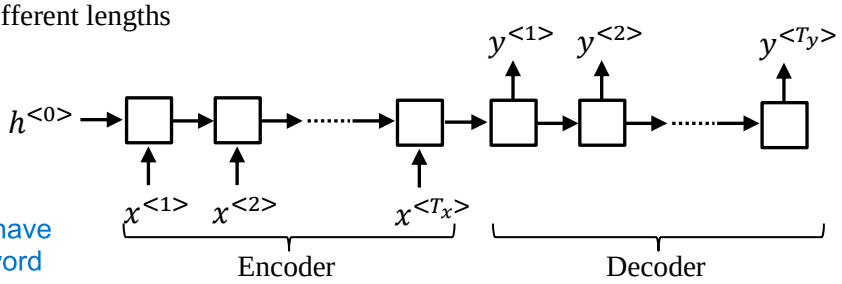
Many-to-many (same input and output length)

- Sequence input, sequence output, with same lengths
- Sequence-to-sequence models
- Example: Part-of-speech tagging / name tagging
  Input: 'Yesterday, Harry Potter met Hermione Granger'
  Output: 'Yesterday, Harry Potter met Hermione Granger'



Many-to-many (different input and output lengths)

- Sequence input, sequence output, with different lengths
- Encoder–decoder architecture
- Example: Machine translation
  Input: 'Where are you from?'
  Output: 'آپ کہاں سے ہیں؟'



input have other order and output have other order/sequence, not word t word translation